

CampusEats — Design Benchmark

CS 543 Web Services — Assignment 2

Project: CampusEats | Service member shown in the service design: Brajakshi Mandloi

1. Capability List

1. Register and manage user accounts.
2. Browse vendors, cafeterias, menus, and food items.
3. Check whether a food item is available.
4. Manage a shopping cart and cart items.
5. Place and track food orders.
6. Create and manage deliveries.
7. Create and record payments.
8. Send order/payment notifications.

2. Services and Data Ownership

CampusEats is divided into four services. Each service owns its own data; no data is shared as an owned resource between services.

Service	Data owned	Member
User Service	Users; User Accounts	Brajakshi Mandloi
Catalogue Service	Vendors; Cafeterias; Menus; Food Items	Brajakshi Mandloi
Order Service	Carts; Cart Items; Orders; Order Items; Deliveries	Brajakshi Mandloi
Payment & Notification Service	Payments; Notifications	Brajakshi Mandloi

Cross-service calls represented in services.drawio

- Order Service → User Service: getUser
- Order Service → Catalogue Service: checkItem
- Order Service → Payment & Notification Service: createPayment

3. Service Contracts

Contracts describe only operations exposed to other services. Internal tables, ID formats, and SQL are hidden.

User Service

Operation	Input	Output	Errors
getUser	user reference	basic user details	USER_NOT_FOUND
createUser	name, contact, account details	created user details	INVALID_USER_DATA
updateUser	user reference, changed details	updated user details	USER_NOT_FOUND; INVALID_DATA

Catalogue Service

Operation	Input	Output	Errors
checkItem	item reference, requested quantity	availability and item details	ITEM_NOT_FOUND; ITEM_UNAVAILABLE
listItems	catalogue/filter request	matching food items	INVALID_FILTER
getMenu	cafeteria/vendor reference	menu and food items	MENU_NOT_FOUND

Order Service

Operation	Input	Output	Errors
addToCart	user reference, item reference, quantity	updated cart summary	USER_NOT_FOUND; ITEM_UNAVAILABLE; INVALID_QUANTITY
placeOrder	user reference, cart/order details	order confirmation/status	CART_EMPTY; ITEM_UNAVAILABLE; PAYMENT_FAILED
getOrder	user reference, order reference	order status and details	ORDER_NOT_FOUND; ACCESS_DENIED

Payment & Notification Service

Operation	Input	Output	Errors
createPayment	order reference, amount, payment request	payment status/reference	INVALID_AMOUNT; PAYMENT_FAILED
getPaymentStatus	order/payment reference	payment status	PAYMENT_NOT_FOUND
sendNotification	recipient, message, event	notification status	INVALID_RECIPIENT; DELIVERY_FAILED

4. Full Specification — placeOrder

Purpose: Create a confirmed CampusEats order from a user's cart after checking item availability and obtaining payment processing.

Inputs: user reference, cart/order details, delivery information, and payment request.

Outputs: order confirmation containing order status, selected items, total amount, delivery status, and payment status.

Success: The order is created only when the user is valid, the cart is not empty, requested items are available, and payment creation succeeds.

Error cases: CART_EMPTY; USER_NOT_FOUND; ITEM_UNAVAILABLE; INVALID_QUANTITY; PAYMENT_FAILED; INVALID_DELIVERY_DATA.

Internal details hidden from callers: The caller does not need to know database table names, primary/foreign-key formats, SQL statements, cart storage, order-item storage, payment persistence, or the internal sequence used to coordinate these steps. Those details remain inside the responsible services.

High-level flow: validate user → inspect cart → check each item with Catalogue Service → calculate/confirm order total → create order internally → request payment from Payment & Notification Service → update order/payment state → return confirmation.

5. Database Schema Design

The schema follows the same service boundaries. Each table belongs to exactly one service.

Service	Tables / main columns
User Service	users(id, name, email, phone); user_accounts(id, user_id, status)
Catalogue Service	vendors(id, name); cafeterias(id, vendor_id, name); menus(id, cafeteria_id, name); food_items(id, menu_id, name, price, available)
Order Service	carts(id, user_id, status); cart_items(id, cart_id, item_ref, quantity); orders(id, user_ref, total, status, delivery_ref); order_items(id, order_id, item_ref, quantity, price); deliveries(id, order_id, address, status)
Payment & Notification	payments(id, order_ref, amount, status); notifications(id, order_ref, recipient, message, status)

The editable ER diagram is provided separately as schema.drawio and the exported diagram as schema.png. The equivalent CREATE TABLE sketch is schema.sql.

6. Service Validation

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled	Fix if needed
User Service	Yes	Yes	Yes	Yes	Yes	None
Catalogue Service	Yes	Yes	Yes	Yes	Yes	None
Order Service	Yes	Yes	Yes	Yes	Mostly	Use only stable catalogue/user/payment contracts; avoid direct data access.
Payment & Notification	Yes	Yes	Yes	Yes	Yes	None